OurMocks

2027 CYCLE

MOCK · PRACTICE · CRACK

CUET UG

Computer Science (Code 308)

Advanced Pandas & Data Operations

Reshaping, Pivoting, GroupBy Split-Apply-Combine, Relational Merging, Joining, Concatenation & Imputation

INSIDE
Concept Notes

INSIDE
Solved Examples

INSIDE
Practice MCQs + Solutions

Aligned to the latest official syllabus · Concept-first explanations

www.ourmocks.com

Syllabus Alignment

Syllabus Section	Core Theoretical & Practical Concepts	CUET Weightage & Exam Focus	Question Typologies
Reshaping & Pivoting	Pivoting data using `df.pivot()` vs `df.pivot_table()`; handling duplicate index-column combinations; aggregation functions (`aggfunc='mean'`, `'sum'`); subtotal margins (`margins=True`); missing value replacement (`fill_value`).	High (2-3 Questions); critical distinction between uniqueness constraint of `pivot()` and aggregation in `pivot_table()`.	Output tracing of 2D pivoted matrices, parameter identification, error diagnosis on duplicate keys.
Aggregation & GroupBy	The Split-Apply-Combine execution model; `df.groupby()`; grouping across single and multiple columns; summary aggregations (`sum`, `mean`, `count`, `std`, `min`, `max`); multi-aggregation using `agg()`; resetting index via `as_index=False`.	Very High (3-4 Questions); core analytical requirement in Section B2 (Informatics Practices).	MultiIndex aggregation tracing, missing value handling in groups, code completion for summary tables.
Merging, Joining & Concat	Database-style joins via `pd.merge()` (`inner`, `left`, `right`, `outer`); column key alignment (`on`, `left_on`, `right_on`); index joins via `df.join()` (`lsuffix`, `rsuffix`); physical stacking via `pd.concat()` along Axis 0 and Axis 1; `ignore_index` semantics.	Extremely High (3-4 Questions); tested across nearly every CUET sitting.	Output shape derivation, column suffix collision tracing, mismatched key join tracing, inner vs outer output comparison.
Sorting & Advanced Cleaning	Sorting data structures via `sort_values()` and `sort_index()`; multi-level sorting; handling null values (`na_position='first'/'last'`); detecting and imputing `NaN` (`isna()`, `dropna()`, `fillna()`, `ffill`, `bfill`); duplicate pruning (`drop_duplicates()`).	High (2-3 Questions); testing in-place mutation, ascending order vectors, and NaN propagation.	Output tracing, boolean mask evaluation, parameter combination analysis, method chaining traces.

CUET HIGH-YIELD EXAMINATION INVARIANTS & FATAL TRAPS

1. **The Reshaping Discrepancy:** `df.pivot()` raises a fatal `ValueError: Index contains duplicate entries, cannot reshape` if any row-column intersection contains multiple values. In sharp contrast, `df.pivot_table()` gracefully handles duplicates by applying an aggregation function (default: `aggfunc='mean'`).
2. **Default Join Typology Mismatch:** `pd.merge()` defaults to `how='inner'`, requiring matching keys in **both** DataFrames. However, `df.join()` defaults to `how='left'`, preserving all rows of the left caller DataFrame!
3. **Suffix Collision in `df.join()`:** Calling `df1.join(df2)` when both DataFrames share an identical column name raises an immediate `ValueError: columns overlap but no suffix specified`. You must supply `lsuffix` and `rsuffix`.
4. **The In-Place Invariant:** By default, `sort_values()`, `dropna()`, `fillna()`, and `drop_duplicates()` return a **new modified copy** and leave the original DataFrame unaltered. Only passing `inplace=True` alters the original object (returning `None`).
5. **GroupBy Laziness:** Calling `df.groupby('col')` returns a lazy `DataFrameGroupBy` object; no numerical computation occurs until an aggregation method (`.mean()`, `.sum()`, etc.) is explicitly invoked!

Advanced Sorting & Multi-Level Ordering Mechanics

Organizing structured data along logical sequences is a prerequisite for both human interpretation and algorithmic processing. Pandas provides two orthogonal sorting paradigms: sorting by **data values** (`sort_values()`) and sorting by **axis labels** (`sort_index()`).

1. Sorting by Values: `df.sort_values()`

The `sort_values()` method rearranges tabular rows based on the values present in one or more specified columns.

```
df.sort_values(
    by,
    axis=0,
    ascending=True,
    inplace=False,
    kind='quicksort',
    na_position='last',
    ignore_index=False
)
```

- `by`: A single column name string or a list of column names strings. If a list is provided, sorting proceeds hierarchically: rows are sorted by the first column; ties are broken by the second column, and so forth.
- `ascending`: Accepts a boolean (`True` for ascending, `False` for descending) or a list of booleans matching the length of `by`. This allows independent sorting directions across columns.
- `na_position`: Controls where missing values (`NaN`) are placed. Options are `'last'` (default) or `'first'`.
- `inplace`: If `False` (default), returns a reordered copy; if `True`, mutates the original DataFrame in place and returns `None`.
- `ignore_index`: If `True`, the resulting row labels are reset to `0, 1, ..., n-1`. If `False` (default), the original row index labels travel with their respective rows.

```
import pandas as pd
import numpy as np

data = {
    'Dept': ['Sales', 'IT', 'Sales', 'HR', 'IT'],
    'EmpName': ['Aman', 'Bina', 'Chirag', 'Divya', 'Esha'],
    'Salary': [55000, 72000, 48000, 61000, 85000]
}
df_emp = pd.DataFrame(data)

# Hierarchical sort: Dept ascending (alphabetical), then Salary descending (numerical)
df_sorted = df_emp.sort_values(by=['Dept', 'Salary'], ascending=[True, False])
print(df_sorted[['Dept', 'EmpName', 'Salary']])
```

CUET HIGH-YIELD DETAIL: INDEX LABEL PRESERVATION IN SORTING

When `df.sort_values()` executes with `ignore_index=False`, the row indices do **not** automatically renumber from 0 to $n-1$. The original row index labels are preserved alongside their data records. To reset the index after sorting, one must explicitly call `.reset_index(drop=True)` or set `ignore_index=True`.

2. Sorting by Axis Labels: `df.sort_index()`

The `sort_index()` method reorders the DataFrame along its coordinate axes based on lexicographical or numerical label order:

- `axis=0` (default): Reorders **rows** based on row index labels.
- `axis=1`: Reorders **columns** alphabetically based on column names.

```
# Sorting columns alphabetically
df_col_sorted = df_emp.sort_index(axis=1)

# Sorting rows in reverse order
df_rev_rows = df_emp.sort_index(axis=0, ascending=False)
```

Reshaping Data: `df.pivot()` vs. `df.pivot_table()`

Tabular data frequently arrives in either "long format" (many rows, few columns, repeating keys) or "wide format" (fewer rows, distinct variables expanded as columns). Transforming between these representations is termed **reshaping**.

1. Basic Reshaping: `df.pivot()`

The `.pivot()` method reshapes data based on column values:

```
df.pivot(index=None, columns=None, values=None)
```

- `index`: Column to use to make new frame's row index.
- `columns`: Column to use to make new frame's columns.
- `values`: Column(s) to use for populating new frame's values.

THE FATAL `PIVOT()` UNIQUENESS RULE

`df.pivot()` requires that each combination of `(index, columns)` must be **strictly unique** across the original dataset! If the dataset contains even a single duplicate pair of `(index, columns)` values, Pandas immediately raises:
`ValueError: Index contains duplicate entries, cannot reshape`
`df.pivot()` CANNOT perform numerical aggregations (it cannot compute sums or averages).

2. Multi-Dimensional Aggregation: `df.pivot_table()`

To reshape datasets containing duplicate index-column combinations, Pandas provides `df.pivot_table()`, which mirrors the behavior of spreadsheet pivot tables.

```
df.pivot_table(
    values=None,
    index=None,
    columns=None,
    aggfunc='mean',
    fill_value=None,
    margins=False,
    dropna=True,
    margins_name='All'
)
```

- `values`: Column to aggregate.
- `index`: Keys to group by on the pivot table row index.
- `columns`: Keys to group by on the pivot table column headers.
- `aggfunc`: Aggregation function or list of functions. Defaults strictly to `'mean'`. Can be `'sum'`, `'count'`, `'min'`, `'max'`, `'std'`, or custom NumPy functions like `np.sum`.
- `fill_value`: Scalar value to replace `NaN` entries in empty intersections (e.g., `fill_value=0`).
- `margins`: Boolean (`True` or `False`). If `True`, adds subtotal and grand total margins labeled `'All'`.

```
sales_data = {
    'Branch': ['North', 'North', 'South', 'North', 'South', 'South'],
    'Product': ['Pen', 'Notebook', 'Pen', 'Pen', 'Notebook', 'Notebook'],
    'Revenue': [1200, 2500, 1500, 1800, 3100, 2900]
}
df_sales = pd.DataFrame(sales_data)

# Aggregating duplicate pairs: Branch 'North' has two 'Pen' sales (1200 and 1800)
pt = df_sales.pivot_table(
    values='Revenue',
    index='Branch',
    columns='Product',
    aggfunc='sum',
    fill_value=0,
    margins=True
)
print(pt)
```

In the resulting pivot table:

- Row `'North'`, Column `'Pen'` contains $1200 + 1800 = 3000$.
- Column `'All'` displays the row total; Row `'All'` displays the column total; bottom-right corner displays the grand total (13000).

Relational Data Combinations: `pd.merge()`, `df.join()` & `pd.concat()`

Combining disparate datasets into a unified analytical view is accomplished via three primary functions, each engineered with distinct alignment invariants and operational semantics.

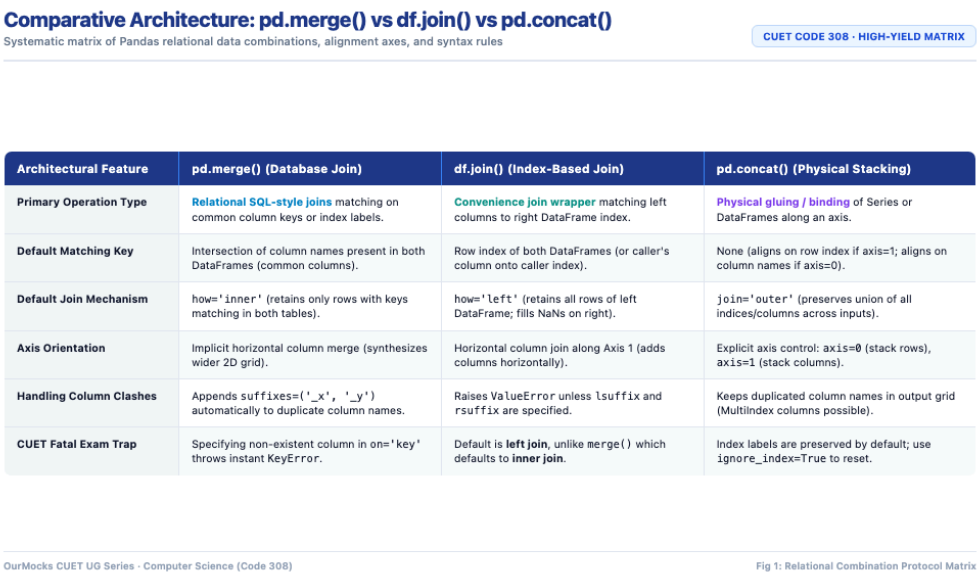


Figure 1: Fig 1: Comparative Architecture — Relational Matrix contrasting pd.merge(), df.join(), and pd.concat() protocols.

1. Database-Style Joins: `pd.merge()`

The `pd.merge()` function implements full relational database joins (equivalent to SQL `INNER JOIN`, `LEFT JOIN`, `RIGHT JOIN`, and `FULL OUTER JOIN`).

```
pd.merge(
    left, right,
    how='inner',
    on=None,
    left_on=None,
    right_on=None,
    left_index=False,
    right_index=False,
    suffixes=('_x', '_y')
)
```

- `left`, `right`: The two DataFrame objects to be merged.
- `how`: Defines the join logic:
  - `'inner'` (Default): Retains only rows whose join keys exist in **both** DataFrames (intersection).
  - `'left'`: Retains all rows from the `left` DataFrame; columns from `right` are filled with `NaN` for non-matching rows.
  - `'right'`: Retains all rows from the `right` DataFrame; columns from `left` are filled with `NaN` for non-matching rows.
  - `'outer'`: Retains all rows from both DataFrames (union); missing key intersections evaluate to `NaN`.
- `on`: Column name(s) to join on. Must exist in both DataFrames. If `None` and `left_on`/`right_on` are not given, merges on the intersection of column names.
- `left_on` / `right_on`: Used when the common key has different column names in the two tables (e.g., `left_on='EmpID'`, `right_on='ID'`).
- `suffixes`: A tuple of string suffixes to append to overlapping column names that are not join keys (default: `('_x', '_y')`).

```
df1 = pd.DataFrame({'EmpID': [1, 2, 3], 'Dept': ['HR', 'IT', 'Sales']})
df2 = pd.DataFrame({'EmpID': [2, 3, 4], 'Project': ['Alpha', 'Beta', 'Gamma']})

# Inner join: EmpID 2 and 3 match
df_inner = pd.merge(df1, df2, on='EmpID', how='inner')
print("Inner Rows:", len(df_inner))  # Output: 2

# Outer join: EmpID 1, 2, 3, 4
df_outer = pd.merge(df1, df2, on='EmpID', how='outer')
print("Outer Rows:", len(df_outer))  # Output: 4
```

2. Index-Based Joins: `df.join()`

The `df.join()` method is a convenience method designed for joining DataFrames primarily along their **row indices**.

```
df_left.join(df_right, on=None, how='left', lsuffix='', rsuffix='', sort=False)
```

- **Key Invariant:** `df.join()` defaults to `how='left'` (unlike `pd.merge()` which defaults to `how='inner'`).
- **Column Overlap Constraint:** If `df_left` and `df_right` share any identical column names, you **must** specify `lsuffix` and `rsuffix`. Failing to do so throws:
  `ValueError: columns overlap but no suffix specified: Index(['Name'], dtype='object')`

```
left_df = pd.DataFrame({'ScoreA': [85, 90]}, index=['Std1', 'Std2'])
right_df = pd.DataFrame({'ScoreB': [78, 92], 'ScoreA': [80, 88]}, index=['Std1', 'Std3'])

# Must supply suffixes because 'ScoreA' overlaps!
joined = left_df.join(right_df, how='inner', lsuffix='_L', rsuffix='_R')
print(joined.columns.tolist())  # ['ScoreA_L', 'ScoreB', 'ScoreA_R']
```

3. Physical Stacking & Gluing: `pd.concat()`

The `pd.concat()` function glues together Series or DataFrames along a designated physical axis.

```
pd.concat(
    objs,
    axis=0,
    join='outer',
    ignore_index=False,
    keys=None,
    verify_integrity=False
)
```

- `objs`: A sequence (list or tuple) of Series or DataFrame objects.
- `axis=0` (Default - Vertical Stacking): Binds rows vertically. Columns are aligned by name. If column sets differ, `NaN` is populated for non-intersecting columns unless `join='inner'`.
- `axis=1` (Horizontal Stacking): Binds columns side-by-side. Rows are aligned by their row index labels.
- `ignore_index`: If `True`, discards existing axis labels and creates a continuous zero-indexed sequence (`0, 1, ..., n-1`).
- `join`: `'outer'` (default, union of non-concatenation axis labels) or `'inner'` (intersection of non-concatenation axis labels).

```
dfA = pd.DataFrame({'Marks': [90, 85]}, index=['R1', 'R2'])
dfB = pd.DataFrame({'Marks': [78, 88]}, index=['R3', 'R4'])

# Vertical Concatenation: Stacking rows
df_vert = pd.concat([dfA, dfB], axis=0, ignore_index=True)
print(df_vert.shape)  # Output: (4, 1), index: 0, 1, 2, 3

# Horizontal Concatenation: Side-by-side columns
dfC = pd.DataFrame({'Age': [17, 18]}, index=['R1', 'R2'])
df_horiz = pd.concat([dfA, dfC], axis=1)
print(df_horiz.shape)  # Output: (2, 2)
```

The GroupBy Lifecycle: Split-Apply-Combine Architecture

In data analysis, computing group-wise summaries (such as the average grade per class, or total revenue per department) follows Hadley Wickham's renowned **Split-Apply-Combine** paradigm.

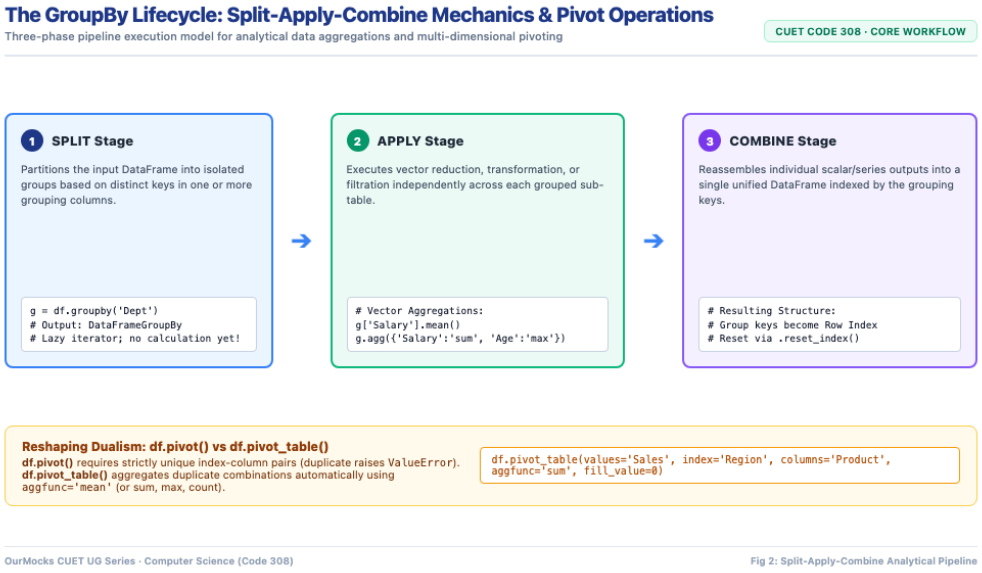


Figure 2: Fig 2: The GroupBy Lifecycle — Split-Apply-Combine Pipeline, Aggregation Vectors, and Index Flattening.

1. Stage 1: The SPLIT Phase

When `df.groupby(by)` is invoked:

- Pandas scans the grouping column(s) and creates an in-memory hash partition mapping unique keys to row index integer positions.
- **Crucial Realization:** `df.groupby()` executes **lazily**. It does not return a DataFrame; it returns an intermediate object of type `<class 'pandas.core.groupby.generic.DataFrameGroupBy'>`.
- No mathematical computation occurs during the Split phase!

```
df_records = pd.DataFrame({
    'Category': ['Hardware', 'Software', 'Hardware', 'Hardware', 'Software'],
    'Item': ['RAM', 'OS', 'SSD', 'GPU', 'Antivirus'],
    'Price': [4500, 8000, 6000, 35000, 2000],
    'Qty': [10, 5, 8, 4, 15]
})

g = df_records.groupby('Category')
print(type(g))  # <class 'pandas.core.groupby.generic.DataFrameGroupBy'>
```

2. Stage 2: The APPLY Phase

In the Apply phase, an aggregation or transformation function is evaluated independently over each group's data subset.

- **Direct Vector Aggregations:** Calling a method directly on the GroupBy object:
  - `g['Price'].mean()`: Computes the mean of `'Price'` per category.
  - `g.sum()`: Sums all numeric columns across each category.
  - Common reduction methods: `.count()`, `.sum()`, `.mean()`, `.median()`, `.std()`, `.min()`, `.max()`, `.first()`, `.last()`.
- **Multi-Column & Multi-Metric Aggregation: `.agg()` / `.aggregate()`:** The `.agg()` method enables heterogeneous aggregations across different columns in a single call using a dictionary mapping:

```
# Heterogeneous aggregations per column
summary = g.agg({
    'Price': ['mean', 'max'],
    'Qty': 'sum'
})
print(summary)
```

3. Stage 3: The COMBINE Phase & Index Realignment

In the Combine phase, Pandas collects the scalar metrics computed for each group and reassembles them into a single unified tabular structure.

- **The Group Key Index Invariant:** By default, the unique values of the grouping column(s) become the **row index** of the resulting output DataFrame!
- If you prefer the grouping key to remain a standard DataFrame column (with a standard 0-indexed integer range), pass `as_index=False` during grouping, or chain `.reset_index()`:

```
# Preserving grouping keys as regular columns
res_col = df_records.groupby('Category', as_index=False)['Price'].mean()
print(res_col)
#      Category    Price
# 0    Hardware  15166.7
# 1    Software   5000.0
```

Advanced Missing Value Handling & Data Cleaning Strategies

Real-world datasets contain anomalies, missing values, duplicates, and inaccurate data types. Developing robust cleaning pipelines is a major theme in CUET.

1. Complete Null Value Detection Protocol

Pandas provides twin boolean mask functions:

- `df.isna()` and `df.isnull()` are completely synonymous and interchangeable.
- `df.notna()` and `df.notnull()` are the exact logical inverse.

```
df_sample = pd.DataFrame({
    'A': [10, np.nan, 30],
    'B': [np.nan, 20, np.nan],
    'C': [1, 2, 3]
})

# Missing value counting patterns:
print(df_sample.isnull().sum())       # Missing count per column
print(df_sample.isnull().sum().sum()) # Total scalar missing count across table (3)
```

2. Systematic Data Pruning: `df.dropna()`

```
df.dropna(axis=0, how='any', thresh=None, subset=None, inplace=False)
```

- `axis=0`: Drops rows with nulls (default); `axis=1`: Drops columns with nulls.
- `how='any'`: Drops the record if **at least one** null value exists.
- `how='all'`: Drops the record **only if all** values in the record are null.
- `thresh=k`: Requires at least k non-null values to keep the record (drops if non-null count $< k$).
- `subset=['ColA', 'ColB']`: Restricts null searching to the specified subset of columns.

3. Data Imputation: `df.fillna()`

- Constant replacement: `df.fillna(0)` or `df.fillna({'A': 0, 'B': 100})`.
- Statistical replacement: `df['A'].fillna(df['A'].mean(), inplace=True)`.
- Directional Imputation:
  - `method='ffill'` (or `pad`): Forward-fills previous observed non-null value downwards.
  - `method='bfill'` (or `backfill`): Backward-fills next observed valid value upwards.

4. Duplicate Record Management: `duplicated()` & `drop_duplicates()`

- `df.duplicated(subset=None, keep='first')`: Returns a boolean Series indicating whether a row is a duplicate.

- `keep='first'` (default): Marks duplicates as `True` except for the first occurrence.
- `keep='last'`: Marks duplicates as `True` except for the last occurrence.
- `keep=False`: Marks **all** duplicates as `True`.

- `df.drop_duplicates(subset=None, keep='first', inplace=False)`: Removes duplicate rows based on specified subset columns.

Comprehensive CUET Code Traces & Analytical Walkthroughs

MULTI-TABLE RELATIONAL MERGING & SUFFIX COLLISION TRACE

Problem: Two departmental DataFrames are defined as follows:

```
import pandas as pd
df_left = pd.DataFrame({
    'ID': [101, 102, 103],
    'Name': ['Amit', 'Bina', 'Charu'],
    'Dept': ['HR', 'IT', 'Finance']
})
df_right = pd.DataFrame({
    'ID': [102, 103, 104],
    'Name': ['Bina K', 'Charu S', 'Deepak'],
    'Rating': [4.5, 4.8, 3.9]
})
res_merge = pd.merge(df_left, df_right, on='ID', how='inner', suffixes=('_Emp', '_Rev'))
```

1. What is the exact value of `res_merge.shape`?
2. List the exact column names present in `res_merge`.
3. What are the values in the row corresponding to `ID == 102`?

Solution:

- **Part 1:** In an inner join on `'ID'`, only IDs present in both tables are retained: $\{101, 102, 103\} \cap \{102, 103, 104\} = \{102, 103\}$ (2 rows). The columns are `'ID'`, left `'Name'`, left `'Dept'`, right `'Name'`, right `'Rating'` (5 columns).
$$\text{res_merge.shape} = (2, 5)$$
- **Part 2:** Because column `'Name'` appears in both DataFrames and is **not** the join key, the suffixes are applied:
$$\text{Columns: } [\text{'ID'}, \text{'Name_Emp'}, \text{'Dept'}, \text{'Name_Rev'}, \text{'Rating'}]$$
- **Part 3:** For `ID == 102`: `'Name_Emp'` is `'Bina'`, `'Dept'` is `'IT'`, `'Name_Rev'` is `'Bina K'`, and `'Rating'` is 4.5.

RESHAPING WITH DUPLICATE COMBINATIONS & AGGREGATION TRACING

Problem: Consider the sales transactions recorded below:

```
import pandas as pd
df_tx = pd.DataFrame({
    'Zone': ['North', 'South', 'North', 'South', 'North'],
    'Quarter': ['Q1', 'Q1', 'Q2', 'Q2', 'Q1'],
    'Sales': [100, 200, 150, 250, 120]
})
```

1. What happens if the code `df_tx.pivot(index='Zone', columns='Quarter', values='Sales')` is executed?
2. What is the output of `df_tx.pivot_table(index='Zone', columns='Quarter', values='Sales', aggfunc='sum', fill_value=0)`?

Solution:

- **Part 1:** The combination `Zone == 'North'` and `Quarter == 'Q1'` appears twice (first with `Sales=100`, then with `Sales=120`). Because `df.pivot()` enforces unique index-column pairs and cannot aggregate, Python raises:
$$\text{ValueError: Index contains duplicate entries, cannot reshape}$$
- **Part 2:** `pivot_table()` aggregates duplicate combinations using `aggfunc='sum'`:
  - For `('North', 'Q1')`: $100 + 120 = 220$.
  - For `('North', 'Q2')`: 150.
  - For `('South', 'Q1')`: 200.
  - For `('South', 'Q2')`: 250.

  Output grid shape is $(2, 2)$ with rows `['North', 'South']` and columns `['Q1', 'Q2']`.

GROUPBY MULTI-METRIC AGGREGATION & MULTIINDEX COLUMN FLATTENING

Problem: An academic institution evaluates test scores across examination centres:

```
import pandas as pd
records = {
    'Centre': ['Delhi', 'Delhi', 'Mumbai', 'Mumbai', 'Delhi'],
    'Score': [85, 90, 78, 88, 95],
    'Hours': [4, 5, 3, 4, 6]
}
df_exam = pd.DataFrame(records)
grp = df_exam.groupby('Centre').agg({'Score': ['mean', 'max'], 'Hours': 'sum'})
```

1. What is the shape of `grp`?
2. What is the value of the mean `'Score'` and total `'Hours'` for `'Delhi'`?

Solution:

- **Part 1:** There are 2 unique centres (`'Delhi'`, `'Mumbai'`), so there are 2 rows. Three aggregations are requested: `'Score'` `mean`, `'Score'` `max`, `'Hours'` `sum`, producing a 2-level MultiIndex across columns.
$$\text{grp.shape} = (2, 3)$$
- **Part 2:** For `'Delhi'`, scores are $85, 90, 95$.
$$\text{Mean Score} = \frac{85 + 90 + 95}{3} = \frac{270}{3} = 90.0$$
Study hours for `'Delhi'` are $4, 5, 6$.
$$\text{Total Hours} = 4 + 5 + 6 = 15$$

Multi-Level Indexing (Hierarchical MultiIndex) & Axis Stacking

In high-dimensional analytical workflows, datasets often possess multiple levels of nested categorization (such as `Region -> State -> City` or `Year -> Quarter`). Pandas supports this natively through **Hierarchical Indexing** (`MultiIndex`).

1. Construction Modes of a MultiIndex

A `MultiIndex` can be generated along rows or columns via several factory methods:

- `pd.MultiIndex.from_tuples(tuples, names=['Outer', 'Inner'])`: Constructs levels from an iterable of tuple coordinates.
- `pd.MultiIndex.from_arrays([array1, array2])`: Combines parallel 1D vectors into layered levels.
- `pd.MultiIndex.from_product([['A', 'B'], [2023, 2024]])`: Computes the Cartesian product of factor arrays.

```
arrays = [
    ['North', 'North', 'South', 'South'],
    ['Delhi', 'Kanpur', 'Chennai', 'Bengaluru']
]
idx = pd.MultiIndex.from_arrays(arrays, names=['Region', 'City'])
df_multi = pd.DataFrame({'Sales': [450, 320, 580, 610]}, index=idx)
print(df_multi)
```

2. Coordinate Navigation & Slicing in MultiIndex

Accessing elements in a hierarchical DataFrame requires tuple coordinate referencing:

- Selecting an outer category: `df_multi.loc['North']` extracts all sub-rows under `'North'` (reducing the MultiIndex by one level).
- Selecting a specific coordinate: `df_multi.loc[('North', 'Delhi')]` extracts the exact row.
- Slicing across levels: Using `pd.IndexSlice` allows cross-sectional slicing: `df_multi.loc[pd.IndexSlice[:, 'Delhi'], :]`.

3. Reshaping via `.stack()` and `.unstack()`

- **`df.unstack(level=-1)`**: Pivots the innermost row index level into column headers (transforms long format to wide format).
- **`df.stack(level=-1)`**: Pivots column headers into innermost row index labels (transforms wide format to long format).
- Both methods accept a `level` argument (integer index or level name) specifying which hierarchical layer to pivot.

Unpivoting DataFrames: `pd.melt()`

While `pivot()` and `pivot_table()` reshape data from "long" to "wide", the inverse transformation—converting wide datasets into tidy long datasets—is executed using `pd.melt()`.

```
pd.melt(
    frame,
    id_vars=None,
    value_vars=None,
    var_name=None,
    value_name='value',
    col_level=None,
    ignore_index=True
)
```

- `id_vars`: Identifier variables (columns to keep intact as row anchors).
- `value_vars`: Columns to unpivot into rows. If `None`, unpivots all columns not listed in `id_vars`.
- `var_name`: String name for the new column storing the unpivoted header names.
- `value_name`: String name for the new column storing the extracted data values.

```
df_grades = pd.DataFrame({
    'Student': ['Ananya', 'Bharat'],
    'Math': [95, 82],
    'Physics': [90, 88],
    'Chemistry': [85, 91]
})

# Melting subject columns into a single 'Subject' column and 'Marks' column
```

```
df_melted = pd.melt(
    df_grades,
    id_vars=['Student'],
    value_vars=['Math', 'Physics', 'Chemistry'],
    var_name='Subject',
    value_name='Marks'
)
print(df_melted)
# Yields 6 rows: each student paired with each subject and mark!
```

Advanced GroupBy Transformations & Filtration

Beyond elementary aggregations (`sum`, `mean`), the GroupBy engine provides two sophisticated analytical operators: `.transform()` and `.filter()`.

1. Group Transformation: `g.transform()`

Unlike aggregation functions (which collapse each group into a single summary scalar, reducing total rows), `.transform()` returns an object of the **exact same size and index** as the input group.

- Every element in a group is mapped to the computed group-level metric.
- Frequently used for calculating percentage of total, group-mean centering, or z-score normalization.

```
df_exp = pd.DataFrame({
    'Dept': ['IT', 'IT', 'HR', 'HR', 'HR'],
    'Salary': [60000, 80000, 45000, 55000, 50000]
})

# Broadcasts the department mean to every employee in that department
df_exp['DeptMean'] = df_exp.groupby('Dept')['Salary'].transform('mean')
# IT employees get 70000; HR employees get 50000!
df_exp['DiffFromMean'] = df_exp['Salary'] - df_exp['DeptMean']
```

2. Group Filtration: `g.filter()`

The `.filter()` method evaluates a boolean predicate function over each group as a whole.

- If the predicate returns `True`, the entire group's rows are retained in the output.
- If the predicate returns `False`, all rows belonging to that group are discarded.

```
# Retaining only departments that have more than 2 employees
df_large_dept = df_exp.groupby('Dept').filter(lambda grp: len(grp) > 2)
# Drops IT (only 2 records); retains HR (3 records)
```

MULTIINDEX UNSTACKING & SHAPE EVOLUTION WALKTHROUGH

Problem: Consider the grouped aggregation output below:

```
import pandas as pd
data = {
    'Dept': ['HR', 'HR', 'IT', 'IT'],
    'Shift': ['Day', 'Night', 'Day', 'Night'],
    'Headcount': [12, 4, 25, 8]
}
df = pd.DataFrame(data)
s = df.groupby(['Dept', 'Shift'])['Headcount'].sum()
df_unstacked = s.unstack()
```

1. What is the type and shape of `s`?
2. What is the type and shape of `df_unstacked`?
3. What is the value of `df_unstacked.loc['IT', 'Night']`?

Solution:

- **Part 1:** `s` is a 1D Series indexed by a 2-level `MultiIndex` (`'Dept'` and `'Shift'`). There are 4 distinct pairs:
  $$\text{type(s)} =< \text{class 'pandas.core.series.Series'} >, \quad \text{s.shape} = (4,)$$
- **Part 2:** `.unstack()` shifts the innermost index level (`'Shift'`) into column headers, producing a 2D DataFrame:
  - Rows: `'Dept'` (`['HR', 'IT']` -> 2 rows).
  - Columns: `'Shift'` (`['Day', 'Night']` -> 2 columns).
  $$\text{type(df_unstacked)} =< \text{class 'pandas.core.frame.DataFrame'} >, \quad \text{shape} = (2, 2)$$
- **Part 3:** Accessing row `'IT'` and column `'Night'` gives the corresponding headcount:
  $$\text{df_unstacked.loc['IT', 'Night']} = 8$$

UNPIVOTING WITH PD.MELT() & SUMMARY RE-AGGREGATION

Problem: A school tracks quarterly sports medal counts:

```
import pandas as pd
df_medals = pd.DataFrame({
    'House': ['Red', 'Blue', 'Green'],
    'Gold': [5, 3, 4],
    'Silver': [2, 6, 3],
```

```
    'Bronze': [4, 2, 5]
})
melted = pd.melt(df_medals, id_vars=['House'], value_vars=['Gold', 'Silver', 'Bronze'],
                 var_name='MedalType', value_name='Count')
res = melted.groupby('House')['Count'].sum()
```

1. What is the shape of `melted`?
2. What is the output of `res['Red']` and `res['Blue']`?

Solution:

- **Part 1:** There are 3 houses and 3 medal categories unpivoted. Total rows $= 3 \times 3 = 9$. Columns are `'House'`, `'MedalType'`, `'Count'` (3 columns).

$$\text{melted.shape} = (9, 3)$$

- **Part 2:** For `'Red'`, medals are Gold (5), Silver (2), Bronze (4):

$$\text{res['Red']} = 5 + 2 + 4 = 11$$

For `'Blue'`, medals are Gold (3), Silver (6), Bronze (2):

$$\text{res['Blue']} = 3 + 6 + 2 = 11$$

OurMocks

Master Synthesis Mindmap: Advanced Pandas & Relational Data Operations

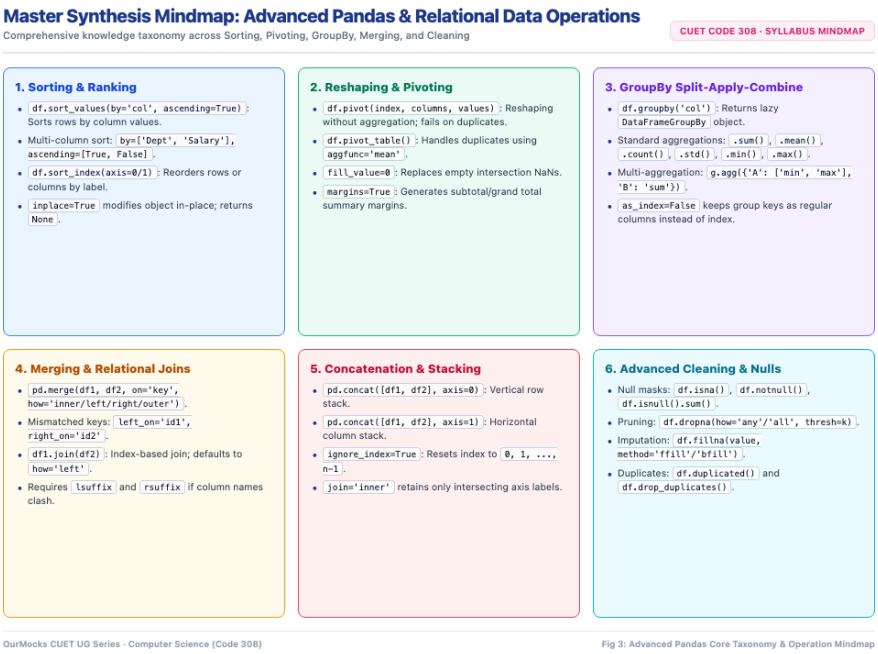


Figure 3: Fig 3: Synthesizer Mindmap — Advanced Pandas Taxonomy, GroupBy Aggregations, Relational Joins, and Pivoting Operations.

OurMocks

Cheatsheet & The Seven Fatal CUET Traps

ADVANCED PANDAS SYNTAX & FUNCTION SIGNATURES CHEATSHEET

- **Sorting Operations:**
  - `df.sort_values(by=['C1', 'C2'], ascending=[True, False], inplace=False, na_position='last')`
  - `df.sort_index(axis=0/1, ascending=True)`: Sorts rows (`axis=0`) or column headers (`axis=1`).
- **Reshaping Operations:**
  - `df.pivot(index='R', columns='C', values='V')`: Strict 1-to-1 mapping; raises `ValueError` on duplicate pairs.
  - `df.pivot_table(values='V', index='R', columns='C', aggfunc='mean', fill_value=0, margins=True)`
- **GroupBy Aggregations:**
  - `df.groupby('K')`: Returns lazy `DataFrameGroupBy`.
  - `df.groupby('K', as_index=False)['V'].mean()`: Keeps `'K'` as a column.
  - `df.groupby('K').agg({'V1': ['min', 'max'], 'V2': 'sum'})`: Multi-column heterogeneous aggregation.
- **Relational Merging & Joining:**
  - `pd.merge(df1, df2, on='Key', how='inner'|'left'|'right'|'outer', suffixes=('_x', '_y'))`
  - `df1.join(df2, how='left', lsuffix='_l', rsuffix='_r')`: Index-based join; defaults to `'left'`.
  - `pd.concat([df1, df2], axis=0|1, join='outer'|'inner', ignore_index=True)`
- **Missing Value & Duplicate Cleaning:**
  - `df.isnull().sum()`: Column-wise null count; `df.isnull().sum().sum()`: Total grid null count.
  - `df.dropna(axis=0|1, how='any'|'all', thresh=k, subset=['C1'])`
  - `df.fillna(val|dict, method='ffill'|'bfill', inplace=False)`
  - `df.drop_duplicates(subset=['C1'], keep='first'|'last'|False, inplace=False)`

THE SEVEN FATAL CUET TRAPS IN ADVANCED PANDAS OPERATIONS

1. **Trap 1: The `pivot()` Duplicate Crash:** Attempting to use `df.pivot()` on data with repeating index-column pairs crashes with `ValueError`. Always use `df.pivot_table()` when data contains duplicates!
2. **Trap 2: `merge()` vs `join()` Default Join Inversion:** `pd.merge()` defaults to `how='inner'`. In stark contrast, `df.join()` defaults to `how='left'`. Misidentifying the default join type in MCQs is the #1 cause of lost marks!
3. **Trap 3: Suffix Collision in `join()`:** If you call `df1.join(df2)` and both DataFrames have a column with the same name, Pandas throws `ValueError: columns overlap but no suffix specified`. You must provide `lsuffix` and `rsuffix`.
4. **Trap 4: Missing Aggregation on GroupBy:** Executing `res = df.groupby('Dept')` and printing `res` prints `<pandas.core.groupby.generic.DataFrameGroupBy object...>`, NOT a table! You must apply an aggregation like `.sum()`.
5. **Trap 5: The In-Place Silent Return:** Calling `df.sort_values(by='A', inplace=True)` mutates `df` and returns `None`. Writing `df = df.sort_values(by='A', inplace=True)` sets `df = None`, destroying your data!
6. **Trap 6: `pd.concat` Axis 0 vs Axis 1:** `axis=0` stacks rows (vertical growth); `axis=1` stacks columns (horizontal growth). Forgetting `ignore_index=True` on `axis=0` produces repeating row indices (`0, 1, 0, 1`).
7. **Trap 7: `dropna()` Default Behavior:** `df.dropna()` without arguments defaults to `axis=0` and `how='any'`, meaning **any** row with even a single `NaN` anywhere is completely removed!

CUET Focused Multiple Choice Questions (Part 1)

Q1. A student executes the following Pandas code on a dataset containing duplicate values for the pair `('City', 'Month')`:
`df.pivot(index='City', columns='Month', values='Rainfall')`
What is the outcome of this operation?

(A) A `ValueError` is raised because `pivot()` cannot aggregate duplicate index-column combinations
(B) Pandas automatically calculates the mean rainfall for duplicate entries
(C) Only the first occurrence of each duplicate combination is retained
(D) The duplicates are overwritten by `NaN` values in the output matrix

Sol. Q1. `df.pivot()` requires that every combination of `(index, columns)` must be strictly unique. If duplicates exist, it raises `ValueError: Index contains duplicate entries, cannot reshape`. To handle duplicate combinations with aggregation, `df.pivot_table()` must be used instead.
Answer: (A)

Q2. Which of the following correctly identifies the default join behaviors of `pd.merge()` and `df.join()`?

(A) `pd.merge()` defaults to `'left'`; `df.join()` defaults to `'inner'`
(B) `pd.merge()` defaults to `'inner'`; `df.join()` defaults to `'left'`
(C) Both functions default strictly to `'inner'` join
(D) Both functions default strictly to `'outer'` join

Sol. Q2. `pd.merge()` defaults to `how='inner'` (matching keys present in both tables). In contrast, `df.join()` is an index-based convenience method that defaults to `how='left'` (retaining all rows from the caller DataFrame).
Answer: (B)

Q3. Consider two DataFrames `df1` and `df2` that both contain a column named `'Score'`. What happens when the expression `df1.join(df2)` is executed without specifying `lsuffix` or `rsuffix`?

(A) The merge succeeds, and the duplicate column in `df2` is silently discarded
(B) Pandas automatically renames the columns to `'Score_x'` and `'Score_y'`
(C) A `ValueError` is raised stating columns overlap but no suffix is specified
(D) The values of column `'Score'` from both tables are added together

Sol. Q3. Unlike `pd.merge()` which automatically appends `('_x', '_y')` to conflicting column names, `df.join()` raises a fatal `ValueError: columns overlap but no suffix specified` if identical column names exist across the two tables without explicit `lsuffix` and `rsuffix` arguments.
Answer: (C)

Q4. Given two DataFrames `dfA` of shape `(3, 4)` and `dfB` of shape `(3, 2)` having identical row indices, what is the shape of the resulting DataFrame from `pd.concat([dfA, dfB], axis=1)`?

(A) `(6, 6)`
(B) `(3, 2)`
(C) `(6, 4)`
(D) `(3, 6)`

Sol. Q4. `pd.concat()` with `axis=1` performs horizontal concatenation (binding columns side-by-side). Since both DataFrames share the same 3 rows, the rows remain 3, and the columns add up: $4 + 2 = 6$. The resulting shape is strictly `(3, 6)`.
Answer: (D)

Q5. Consider the DataFrame `df`:

```
df = pd.DataFrame({
    'Dept': ['HR', 'IT', 'HR', 'IT'],
    'Salary': [50000, 70000, 60000, 65000]
})
df_res = df.sort_values(by=['Dept', 'Salary'], ascending=[True, False])
```

What is the resulting order of the `'Salary'` column in `df_res`?

(A) `[60000, 50000, 70000, 65000]`
(B) `[50000, 60000, 65000, 70000]`
(C) `[70000, 65000, 60000, 50000]`
(D) `[65000, 70000, 50000, 60000]`

Sol. Q5. Sorting proceeds hierarchically: first by `'Dept'` ascending (`'HR'` before `'IT'`). Within `'HR'`, `'Salary'` is sorted descending $(60000, 50000)$. Within `'IT'`, `'Salary'` is sorted descending $(70000, 65000)$. Thus, the complete order of salaries is `[60000, 50000, 70000, 65000]`.
Answer: (A)

CUET Focused Multiple Choice Questions (Part 2)

Q6. What is the return type of executing `df.groupby('Department')` before applying any aggregation function?

(A) `pandas.core.frame.DataFrame`
(B) `pandas.core.groupby.generic.DataFrameGroupBy`
(C) `pandas.core.series.Series`
(D) `dict` of DataFrames

Sol. Q6. `df.groupby()` is a lazy partitioning operation. It does not compute summaries or return a DataFrame immediately; it returns an intermediate `DataFrameGroupBy` object that awaits an aggregation or transformation method.
Answer: (B)

Q7. To compute the average of column `'Salary'` and the maximum of column `'Age'` simultaneously across departments, which syntax is correct?

(A) `df.groupby('Dept')['Salary'].mean() & df.groupby('Dept')['Age'].max()`
(B) `df.groupby('Dept').agg(['mean', 'max'])`
(C) `df.groupby('Dept').agg({'Salary': 'mean', 'Age': 'max'})`
(D) `df.groupby('Dept').pivot_table(Salary='mean', Age='max')`

Sol. Q7. Heterogeneous column aggregations in Pandas are achieved by passing a dictionary mapping to `.agg()`, where keys represent column names and values specify the corresponding reduction functions: `df.groupby('Dept').agg({'Salary': 'mean', 'Age': 'max'})`.
Answer: (C)

Q8. When grouping a DataFrame using `df.groupby('Section')`, how can you ensure that `'Section'` remains a regular column in the output instead of becoming the row index?

(A) Pass `keep_index=True` in `df.groupby()`
(B) Pass `index=False` in `df.groupby()`
(C) Use `df.groupby('Section').drop_index()`
(D) Pass `as_index=False` in `df.groupby('Section')`

Sol. Q8. By default, the grouping column becomes the row index of the aggregated output. Setting `as_index=False` in `groupby()` prevents this and maintains the grouping column as a regular DataFrame column with a standard integer index.
Answer: (D)

Q9. Consider the call:
`df.pivot_table(values='Sales', index='Region', columns='Product', aggfunc='sum', fill_value=0)`
What is the purpose of the `fill_value=0` parameter?

(A) It replaces `NaN` with `0` for combinations of `Region` and `Product` that have no sales records
(B) It sets any negative sales values in the original dataset to `0`
(C) It forces the grand total margins to start from `0`
(D) It initializes the DataFrame with zero rows before pivoting

Sol. Q9. When reshaping via `pivot_table()`, if a specific intersection of `Region` and `Product` has no records in the original data, Pandas would normally place `NaN`. Supplying `fill_value=0` populates these empty cells with `0` instead.
Answer: (A)

Q10. Which Pandas expression returns the total number of missing (`NaN`) values present in **each column** of a DataFrame `df`?

(A) `df.count()`
(B) `df.isnull().sum()`
(C) `df.isna().count()`
(D) `df.dropna().size`

Sol. Q10. `df.isnull()` generates a boolean mask where `NaN` cells evaluate to `True` (1) and valid cells evaluate to `False` (0). Calling `.sum()` aggregates along axis 0 (rows), returning the exact count of missing values per column.
Answer: (B)

CUET Focused Multiple Choice Questions (Part 3)

Q11. Consider the Series `s`:

```
import pandas as pd
import numpy as np
s = pd.Series([10, np.nan, np.nan, 40, np.nan])
s_filled = s.fillna(method='ffill')
```

What is the resulting value of `s_filled[2]`?

(A) `NaN` (B) `0.0`
(C) `10.0` (D) `40.0`

Sol. Q11. `method='ffill'` (forward fill) propagates the last observed non-null value forward downwards. At index 0, the value is 10. This value 10 is propagated to index 1 and index 2. Therefore, `s_filled[2]` evaluates strictly to `10.0`.
Answer: (C)

Q12. Consider the DataFrame `df`:

```
df = pd.DataFrame({
    'Dept': ['IT', 'HR', 'IT', 'Sales'],
    'Bonus': [5000, 4000, 6000, 3000]
})
df_unique = df.drop_duplicates(subset=['Dept'], keep='last')
```

Which bonus value is retained for the `'IT'` department in `df_unique`?

(A) `5000` (B) `5500`
(C) `NaN` (D) `6000`

Sol. Q12. The parameter `keep='last'` instructs Pandas to keep the last occurrence among duplicates and discard earlier ones. For `'IT'`, the occurrences are 5000 (first) and 6000 (last). Thus, 6000 is retained.
Answer: (D)

Q13. When merging two DataFrames `dfA` and `dfB`, the common key is named `'EmpCode'` in `dfA` and `'EmployeeID'` in `dfB`. Which syntax correctly performs an inner merge?

(A) `pd.merge(dfA, dfB, left_on='EmpCode', right_on='EmployeeID', how='inner')`
(B) `pd.merge(dfA, dfB, on=['EmpCode', 'EmployeeID'])`
(C) `dfA.join(dfB, left_on='EmpCode', right_on='EmployeeID')`
(D) `pd.concat([dfA, dfB], on=['EmpCode', 'EmployeeID'])`

Sol. Q13. When the join key column has different names across the two tables, `left_on` and `right_on` must be explicitly specified in `pd.merge()`. Passing both to `on` raises a `KeyError`.
Answer: (A)

Q14. Given `df1` with row index `['A', 'B', 'C']` and `df2` with row index `['B', 'C', 'D']`, what is the row index of `pd.concat([df1, df2], axis=1, join='inner')`?

(A) `['A', 'B', 'C', 'D']` (B) `['B', 'C']`
(C) `['A', 'D']` (D) `[0, 1]`

Sol. Q14. In `pd.concat` with `axis=1` (horizontal stacking), `join='inner'` takes the intersection of the row index labels. The intersection of `{'A', 'B', 'C'}` and `{'B', 'C', 'D'}` is strictly `['B', 'C']`.
Answer: (B)

Q15. Consider the DataFrame `df`:

```
df = pd.DataFrame({
    'City': ['Goa', 'Goa', 'Pune', 'Pune'],
    'Sales': [100, np.nan, 200, 300]
})
res = df.dropna(subset=['Sales']).groupby('City')['Sales'].sum()
```

What is the resulting value of `res['Goa']`?

(A) `NaN` (B) `0.0`
(C) `100.0` (D) `300.0`

Sol. Q15. `df.dropna(subset=['Sales'])` removes the second row where `'Sales'` is `NaN`. For `'Goa'`, only the first row with `Sales=100` remains. The subsequent groupby sum for `'Goa'` evaluates strictly to 100.0.
Answer: (C)